DATABASE DESIGN PORTFOLIO SAMPLE

Relational Schema Design and Registration Modeling

This sample presents a normalized schema for a college registration system and shows how entity design, foreign-key enforcement, and supporting indexes can be translated into a readable employer-facing artifact.

FOCUS

Entity modeling, relationship structure, key constraints, and enrollment logic.

AUDIENCE

Database support, systems, reporting, implementation, and technically adjacent operations roles.

FORMAT

Presentation wrapper designed for browser review and clean print-to-PDF export.

WHAT THIS DEMONSTRATES

Database thinking translated into a compact, readable technical sample.

Normalized Table Design

The schema separates instructors, students, classes, and registrations into purposeful entities with clear keys.

Relationship Management

The registration table models many-to-many enrollment cleanly while preserving referential integrity.

Constraint Awareness

Foreign keys and indexes reinforce the design and support consistency, lookup speed, and maintainability.

Operational Reporting

The example roster query shows how a schema can move from design into real reporting use.

CODE SAMPLE

Sanitized public portfolio copy

```
/*
  Sanitized public portfolio copy: relational schema design sample for a
  college registration system.

  What this sample demonstrates:
  - normalized entity design
  - primary and foreign key enforcement
  - many-to-many registration modeling
  - naming consistency and supporting indexes
*/

DROP DATABASE IF EXISTS college_registration_demo;
CREATE DATABASE college_registration_demo;
USE college_registration_demo;

CREATE TABLE instructors (
    instructor_id VARCHAR(5) PRIMARY KEY,
    first_name VARCHAR(30) NOT NULL,
    last_name VARCHAR(30) NOT NULL,
    phone VARCHAR(15)
);

CREATE TABLE students (
    student_id VARCHAR(5) PRIMARY KEY,
    first_name VARCHAR(25) NOT NULL,
    last_name VARCHAR(25) NOT NULL,
    major VARCHAR(40) NOT NULL
);

CREATE TABLE classes (
```

```
    class_code VARCHAR(5) PRIMARY KEY,
    section_number VARCHAR(5) NOT NULL,
    instructor_id VARCHAR(5) NOT NULL,
    location VARCHAR(40) NOT NULL,
    CONSTRAINT fk_classes_instructor
        FOREIGN KEY (instructor_id) REFERENCES instructors (instructor_id)
);

CREATE TABLE registrations (
    class_code VARCHAR(5) NOT NULL,
    student_id VARCHAR(5) NOT NULL,
    enrolled_on DATE NOT NULL,
    PRIMARY KEY (class_code, student_id),
    CONSTRAINT fk_registrations_class
        FOREIGN KEY (class_code) REFERENCES classes (class_code),
    CONSTRAINT fk_registrations_student
        FOREIGN KEY (student_id) REFERENCES students (student_id)
);

CREATE INDEX idx_classes_instructor
    ON classes (instructor_id);

CREATE INDEX idx_registrations_student
    ON registrations (student_id);

-- Structural verification queries.
SHOW CREATE TABLE instructors;
SHOW CREATE TABLE students;
SHOW CREATE TABLE classes;
SHOW CREATE TABLE registrations;

-- Example reporting query: active roster by class section.
SELECT
    c.class_code,
    c.section_number,
    CONCAT(i.last_name, ', ', i.first_name) AS instructor_name,
    COUNT(r.student_id) AS enrolled_students
```

```
FROM classes AS c
INNER JOIN instructors AS i
    ON i.instructor_id = c.instructor_id
LEFT JOIN registrations AS r
    ON r.class_code = c.class_code
GROUP BY c.class_code, c.section_number, i.last_name, i.first_name
ORDER BY c.class_code, c.section_number;
```